Lecture 10

18 October 2026 19:54

When moving from the basic Bigram model to Self-Attention, Karpathy adjusts three key hyperparameters:

python
Before (Bigram) # Now (Attention / GPT)
max_iters = 3000 --> max_iters = 5000
eval_interval = 300 --> eval_interval = 500
learning_rate = 1e-2 --> learning_rate = 1e-3 (or 3e-4)
Here is why each was changed and how they work together:

1. learning_rate: from 1e-2 (0.01) ⟶ 1e-3 (0.001)

- **Why it changed:**
  - **Bigram was super simple** (just a lookup table / single layer). You could use a massive learning rate (1e-2) and take giant steps without breaking anything.
  - **Self-Attention is a deeper, delicate network** with matrix multiplications, softmax, and multiple projections. If the learning rate is too high (0.01), the gradients explode, the weights overshoot, and the loss diverges.
- **Analogy:**
  Walking on a flat field vs. walking on a rocky mountain ridge. On a mountain ridge (deep network), taking giant leaps (1e-2) will make you fall off a cliff. You must take smaller, careful steps (1e-3).

2. max_iters: from 3000 ⟶ 5000

- **Why it changed:**
  - Because we lowered the learning rate (taking smaller steps), the model needs **more total steps** to travel down the loss curve.
  - Also, attention models are much smarter and have more capacity to learn. Giving it 5,000 steps allows it to find subtle language patterns that Bigram could never capture.

3. eval_interval: from 300 ⟶ 500

- **Why it changed:**
  - estimate_loss() takes time because it runs eval_iters = 200 batches on train and 200 batches on val without gradients to get a clean average.
  - If you have 5000 iterations and check every 50 steps, training would be super slow because you'd spend most of your time calculating eval loss!
  - Setting it to 500 means it pauses to log progress roughly **10 times** across the whole training run (step 0, 500, 1000, ... 4500), striking a perfect balance between monitoring progress and fast training.

In Short:

We take **smaller, gentler steps** (learning_rate = 1e-3), so we need **more steps to get there** (max_iters = 5000), and we **check our progress every 500 steps** (eval_interval = 500) to avoid wasting time!

So, now the after training, the loss in both train data and val data is changed from roughly 2.5. something down to 2.40
But till the text is not good

Now, making multi-head attention :

1. Why do we need it ?

For now, we have single head of 16 and 16 size, so 1 head will understand let's say grammar and other head understand the verbs, and then we are combining the both outputs together.

Hence, multiple heads will work on different patterns and the model will train faster.

2. The code :

Class multiHeadAttention :

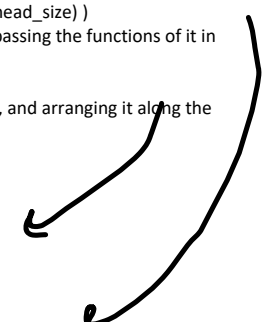
```
def __init__(self, num_heads, head_size) :
    super().__init()
    self.heads =  nn.ModuleList( [ Head ( head_size )  for _  in range(num_heads) ] )

def forward(self, x):
    return torch.cat ( [ h(x) for h in self.heads ], dim=-1 )
```

1. This will create a list of different 4 total heads and pass those 4 heads in the single head of self-attention (Head(head_size))

And this list will be passed to the nn.Module or we can say the nn.ModuleList do the work of creating the list and then passing the functions of it in the module

2. Then we will get the combine output of those separate heads with their separate output , and then concatination, and arranging it along the channel dimension
   Like this :



Step 1: What is h and what does h(x) do?

In __init__, we created 4 distinct Head objects:

python
self.heads = nn.ModuleList([Head(8), Head(8), Head(8), Head(8)])

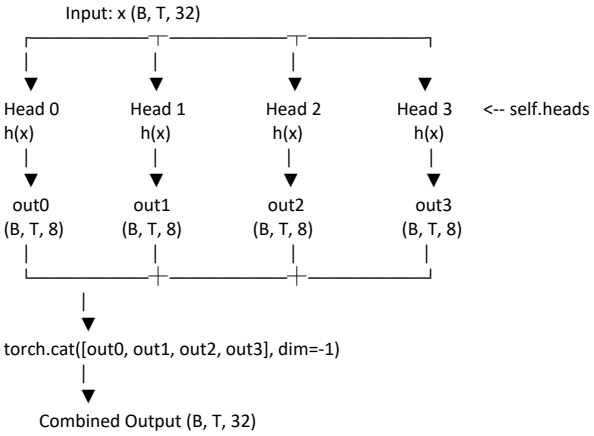
Let's call them **Head_0**, **Head_1**, **Head_2**, and **Head_3**. Each has its own independent weights (W_q, W_k, W_v).
When you write [h(x) for h in self.heads], it is just a Python list comprehension that does:

```
out0 = self.heads[0](x)   # Head 0 processes x  --> shape: (B, T, 8)
out1 = self.heads[1](x)   # Head 1 processes x  --> shape: (B, T, 8)
out2 = self.heads[2](x)   # Head 2 processes x  --> shape: (B, T, 8)
out3 = self.heads[3](x)   # Head 3 processes x  --> shape: (B, T, 8)

# The list comprehension produces:
head_outputs = [out0, out1, out2, out3]
```

Every head takes the **same input x** of shape (B, T, 32) and produces an output of shape **(B, T, 8)**.

Step 2: The Flow Diagram

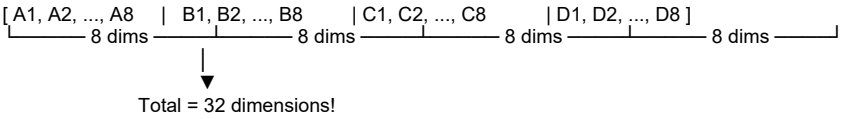


Step 3: How does torch.cat(..., dim=-1) glue them together?

dim=-1 means the **last dimension** (the channel/feature dimension).
The batch size B and sequence length T stay untouched. We simply glue the feature vectors side-by-side!
Let's look at a single token at position *t*:

- Head 0 produces 8 numbers: [A1, A2, A3, A4, A5, A6, A7, A8]
- Head 1 produces 8 numbers: [B1, B2, B3, B4, B5, B6, B7, B8]
- Head 2 produces 8 numbers: [C1, C2, C3, C4, C5, C6, C7, C8]
- Head 3 produces 8 numbers: [D1, D2, D3, D4, D5, D6, D7, D8]

When torch.cat(..., dim=-1) runs:



main Part V. IMP

The main Multi head self-attention :

1. The Core Idea: Divide and Conquer 🧠

Instead of having **1 big head** of size 32:

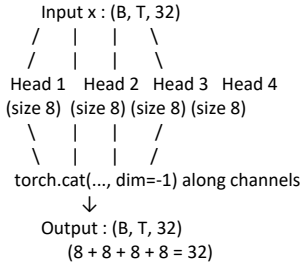
- We create **4 smaller heads**, each of size 8: $head_size = \frac{n_embd}{num_heads} = \frac{32}{4} = 8$

Why?

- **Head 1 (size 8):** Can specialize in tracking nouns and verbs.
- **Head 2 (size 8):** Can specialize in finding rhyming patterns.
- **Head 3 (size 8):** Can specialize in punctuation and quotes.
- **Head 4 (size 8):** Can specialize in long-distance pronouns ("he", "she", "it").

They all look at the same sequence from **different perspectives** simultaneously!

2. The Shape Transformation 📐



The output shape is **(B, T, 32)** — exactly matching n_embd!

3. The MultiHeadAttention Class

Here is how Karpathy writes it:

```
python
class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """
    def __init__(self, num_heads, head_size):
        super().__init__()
        # Create a list of independent Head instances
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
    def forward(self, x):
        # Run every head on x in parallel, and concatenate results on the channel dimension
        return torch.cat([h(x) for h in self.heads], dim=-1)
```

Why nn.ModuleList instead of a plain Python [Head(...) for ...]?
If you use a regular Python list [], PyTorch won't know the layers inside have weights, so .to(device) and optimizer training won't find them! nn.ModuleList tells PyTorch: *"These are neural network layers to track and train."*

4. How ~~it plugs into BigramLanguageM~~odel

```
In __init__:
# Before (single head):
# self.sa_head = Head(n_embd)

# Now (multi-head): 4 heads of size 8 = 32
self.sa_heads = MultiHeadAttention(4, n_embd // 4)
self.lm_head  = nn.Linear(n_embd, vocab_size)


In forward():
x = tok_emb + pos_emb   # (B, T, 32)
x = self.sa_heads(x)    # (B, T, 32)  <-- all 4 heads communicate and combine!
logits = self.lm_head(x)# (B, T, vocab_size)
```

In Tensor Shapes:

$Tensor\ 0: (B,T,\mathbf{8})$ $Tensor\ 1: (B,T,\mathbf{8})$ $Tensor\ 2: (B,T,\mathbf{8})$ $Tensor\ 3: (B,T,\mathbf{8})$

$\xrightarrow[cat(.,..dim=-1)]{}$ $(B,T,\mathbf{8}+\mathbf{8}+\mathbf{8}+\mathbf{8}) = (B,T,\mathbf{32})$

And hence the loss drops from 2.40 to 2.30 roughly

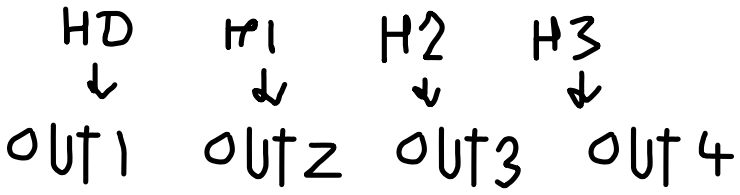
Now, the FeedForward class :

We are in a hurry to calculate the logits(which is the output), but we need the new tokens generated by each multi-head to communicate with each other.

What does communication with each other MEANS ?

Each head has its own new output or let's say predicted next token by performing the following operation :

different Query finds answer or ask question to the key ----> and -----> their output gets combine with possible related output.



We need each head to talk to other head what they have found and study other head patterns

Andrej Karpathy summarizes the FeedForward class with one famous line:

"Self-Attention is communication; Feed-Forward is computation (thinking)."

1. The Analogy 🏠

- **Self-Attention:** Tokens turn to each other and exchange information (like students in a group discussion collecting notes).
- **Feed-Forward:** Each token goes back to its own desk and **thinks** about what it just learned, digesting the new information individually.

2. What is it in PyTorch?

It is simply a small **Multi-Layer Perceptron (MLP)** applied to every token independently:

Perceptron : V. IMP DeF

A perceptronis the simplest form of an artificial neural network, functioning as a single artificial neuron that serves as a linear binary classifier for supervised learning. Invented by Frank Rosenblattin 1957, it mimics biological neurons by taking multiple input values, multiplying them by corresponding weights, adding a bias, and applying a step activation function to produce a binary output(typically 0 or 1).

From <https://search.brave.com/search?q=Perceptron+meaning+in+ml&source=web&conversation=09b355586abd4591ef6c52d3eff4dabd7b75>

As our neurons not just examine the problem and finds the solution, but also take information of the other neurons like what they think of the problem, take those information and also learn from it

```
class FeedForward(nn.Module):
  """ a simple linear layer followed by a non-linearity """

  def __init__(self, n_embd):
    super().__init__()
    self.net = nn.Sequential(
      nn.Linear(n_embd, 4 * n_embd),  # Expand: 32 --> 128
      nn.ReLU(),                  # Non-linearity
      nn.Linear(4 * n_embd, n_embd),  # Project back: 128 --> 32
    )

  def forward(self, x):
    return self.net(x)
```

*(Note: Initially in the video, Karpathy writes just nn.Linear(n_embd, n_embd) and nn.ReLU(), but then immediately expands it by a factor of 4 (4 * n_embd) to match the original Transformer paper).*

1. nn.Sequential use :

In PyTorch, **torch.nn.Sequential** is a container module used to chain layers together sequentially. The output of one layer is automatically passed as the input to the next layer in the order they are defined.

Key Reasons to Use nn.Sequential

1. Eliminates Boilerplate Code

Without nn.Sequential, you must define every layer in __init__ and explicitly pass data through each one inside forward(). With nn.Sequential, PyTorch handles the forward pass for you.
Without nn.Sequential:

```
python
import torch.nn as nn
class MLP(nn.Module):
  def __init__(self):
    super().__init__()
    self.fc1 = nn.Linear(784, 128)
    self.relu = nn.ReLU()
    self.fc2 = nn.Linear(128, 10)

  def forward(self, x):
    x = self.fc1(x)
    x = self.relu(x)
    x = self.fc2(x)
    return x
```

With nn.Sequential:

```
python
model = nn.Sequential(
  nn.Linear(784, 128),
  nn.ReLU(),
  nn.Linear(128, 10)
)

# No custom class or forward() method required!
```

2. ReLU use :

ReLU stands for **Rectified Linear Unit**. It is the most widely used **activation function** in modern deep learning models.

2. Why Use ReLU?

A. Introduces Non-Linearity

Without an activation function, stacking linear layers is mathematically equivalent to just one single linear layer: $W_2\left(W_1x + b_1\right) + b_2 = W^{'}x + b^{'}$ ReLU introduces the non-linearity required for a network to learn complex patterns and functions.

If the weights are added to the next weights then = it is called as Linear,
but we need the weight to add to the new wights for complex calculations,
Which is called as, Non-Linearity ($W_2\left(W_1x + b_1\right) + b_2 = W^{'}x + b^{'}$ R)

3. Why expand by $4\times$ (4 * n_embd)?

- In the original paper *"Attention Is All You Need"*, the inner dimension of the feed-forward network is always **4 times wider** than the embedding size.
- For n_embd = 32, the inner layer expands to 128: $Input$: $32 \xrightarrow{Linear} 128 \xrightarrow{ReLU} 128 \xrightarrow{Linear} 32$
- **Why?** Expanding into a higher-dimensional space gives the network more "room" to create complex, non-linear combinations of features before projecting back down to 32.

4. How Attention and FeedForward work together

In your model's pipeline, they form a tag-team:

Input Tokens: x
▼
Multi-Head Attention ⟶ Tokens talk to each other & share context
▼
Feed-Forward ⟶ Each token reflects & processes what it learned
▼
Logits ⟶ Predict the next character!

Self-Attention gathers the clues; Feed-Forward solves the riddle!

V IMP :
Question : why we expand it to 128, part of neural network

Ans :

Let's break this down from scratch—no complicated math, just plain intuition!

Step 1: What do the 32 numbers represent?

Every token is described by a vector of **32 numbers** (n_embd = 32).

Think of these 32 numbers as **32 basic clues** about the token:

- Number 1: *"Is it a verb?"*
- Number 2: *"Is it capitalized?"*
- Number 3: *"Is it related to people?"*
- ... up to 32 basic facts.

Step 2: The Problem: Complex concepts need *combinations* of clues

In real language, predicting the next word requires understanding **complex combinations** of clues, such as:

> *"Is it a **verb** AND **past tense** AND **referring to a plural subject**?"*

With only 32 numbers, the model has very little room to test for all these different combinations simultaneously.

It needs a **larger "scratchpad"** or workspace to think.

Step 3: Expanding to 128 (nn.Linear(32, 128))

When we expand from 32 to 128, the network creates **128 new specialized "detectors"**:

[Equation]
Each of the 128 neurons looks at the 32 basic clues and asks its own unique question:

- Detector #1 might ask: *"Is clue 3 positive AND clue 7 negative?"*

- Detector #2 might ask: *"Is this an action word happening in a royal court?"*
- Detector #128 might ask: *"Are we inside a question sentence?"*

By expanding to 128, we give the model **4 times more capacity** to consider different patterns.

Step 4: The Magic of nn.ReLU() (The ON/OFF Switch)

After expanding to 128 numbers, we pass them through **ReLU**: $ReLU(x) = max(0, x)$
ReLU is simply a switch:

- If a detector's question was **YES** (positive number): it **stays ON** and passes through.
- If a detector's question was **NO** (negative number): it becomes 0 **(turned OFF)**.

IMP

This is what makes neural networks so powerful! Out of the 128 detectors, only the ones relevant to this specific word will turn **ON**.

Step 5: Compressing back to 32 (nn.Linear(128, 32))

Now that the 128 detectors have done their thinking and filtered what's important, we can't keep a 128-dimensional vector, because the rest of our model (and the next attention layers) expects vectors of size **32**.
So the second linear layer **summarizes** the results:

[Equation]
FOR EXAMPLE :

The Kitchen Analogy

Think of it like cooking:

1. **Start (32):** You bring **32 ingredients** into the kitchen packed tightly in a bag.
2. **Expand (128):** You spread them out onto a **large table (128 spaces)** so you have room to chop, mix, and combine different ingredients together.
3. **ReLU:** You taste-test and **throw away** what doesn't taste good (turn them to 0), keeping only the successful combinations.
4. **Compress (32):** You plate the final dish into a **compact meal (32)** ready to serve!

Summary

[32 raw clues] ——> Expand to 128 combinations
——> ReLU filters the good combinations
——> Compress back to 32 refined clues!

This **Expand** → **Filter** → **Compress** pattern is the engine of almost every neural network in existence!

Changes in the bigram language model :

1. __init__() part :

self.ffwd = FeedFoward(n_embd)

2. In forward :

x = Self.ffwd(x)

Summary of the Flow:

[Equation]

So, now the loss is 2.20 something